

課題⑤ iptables によるポート転送

目的

iptables を用いたパケットの制御の実習を行う。パケットを分析することによりポート転送の方法を確認する。

事前準備

1. server と router の VM を起動する。
2. iptables コマンドについて調査する。

手順

1. router で dropbear が起動していないことを確認し、起動している場合は停止する。

図 1 は router で dropbear が起動していないことを確認する様子である。

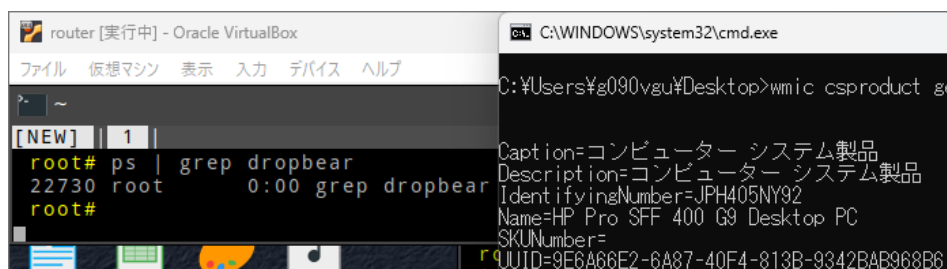


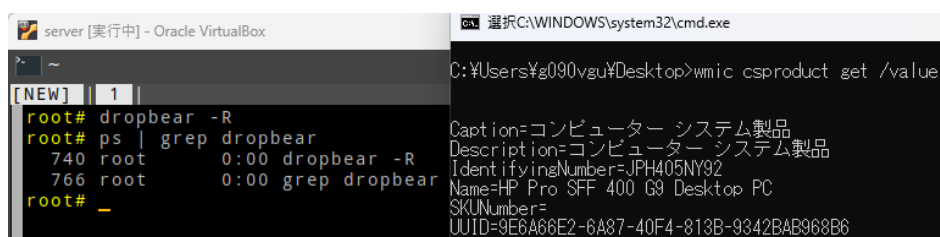
図 1 server で dropbear が起動していないことの確認

ps コマンドで プロセス一覧を取得し、その結果を grep コマンドで dropbear という文字列を含むものに限定している。フィルタリングされたプロセス一覧には、grep dropbear(プロセス一覧からの文字列検索に用いたコマンド) 以外のコマンドによるプロセスは表示されていない。したがって、router において dropbear が起

動していないことが確認できる。

2. server で dropbear を -R オプション付きで起動し，起動を確認する。

図 2 は server で dropbear を -R オプション付きで起動し，起動を確認する様子である。



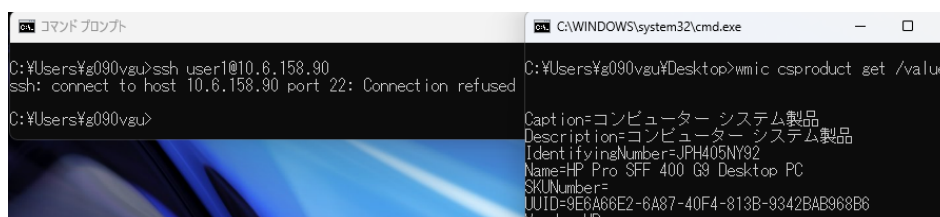
```
server [実行中] - Oracle VirtualBox
[NEW] 1
root# dropbear -R
root# ps | grep dropbear
740 root    0:00 dropbear -R
766 root    0:00 grep dropbear
root# _
```

図 2 server で dropbear 起動確認

ここではプロセス一覧において dropbear -R というコマンドで起動されたプロセスが表示されていることから，dropbear が-R オプション付きで起動していることが確認できる。

3. デスクトップ PC(Windows) から router へ ssh で一般権限ユーザでログインできないことを確認する。

図 3 はデスクトップ PC から router への ssh 接続に失敗する様子である。



```
コマンドプロンプト
C:\Users\%g090vgu>ssh user1@10.6.158.90
ssh: connect to host 10.6.158.90 port 22: Connection refused
C:\Users\%g090vgu>
```

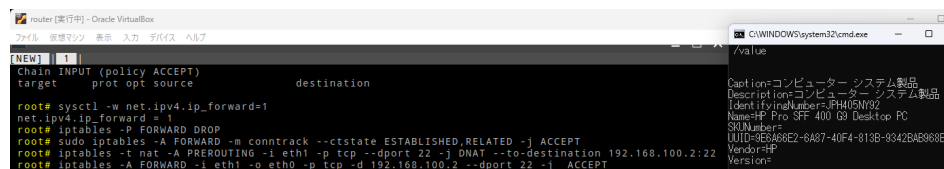
図 3 デスクトップ PC から router への ssh 接続に失敗する様子

ssh: connect to host 10.6.158.90 port 22: Connection refused というエラーメッセージが表示されている。手順 1 で確認したとおり，router においては dropbear のプロセスが起動されていないためである。

4. router の組織外側インターフェースに対して以下の iptables 設定を行う（必要なら sysctl で転送を有効化：net.ipv4.ip_forward=1）：

- ① TCP 22 番ポート宛てのパケットを server の TCP 22 番ポートへ転送する（例：PREROUTING で DNAT を設定）。
- ② TCP のそれ以外のポート宛てアクセスは転送しない（DROP/REJECT を設定）。

図 4 は手順 4 の要件を満たすような iptables の設定を行うコマンドの実行結果である。各コマンドの説明については手順 10 で行う

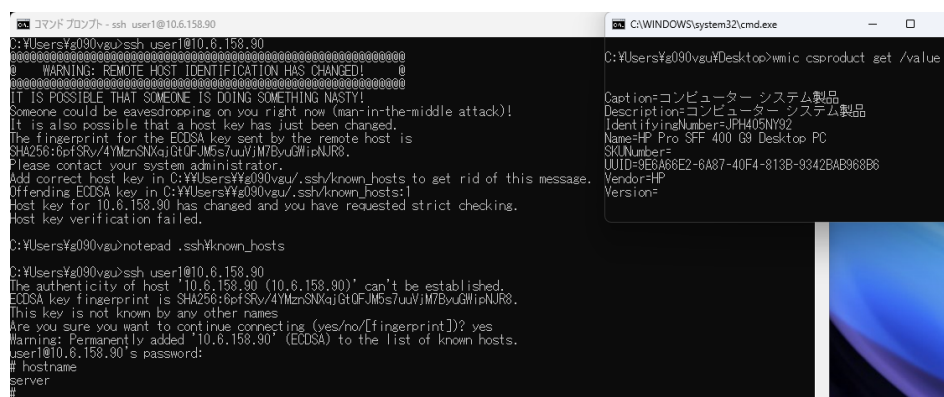


```
root@router:~# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@router:~# iptables -P FORWARD DROP
root@router:~# sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
root@router:~# iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 22 -j DNAT --to-destination 192.168.100.2:22
root@router:~# iptables -A FORWARD -i eth1 -o eth0 -p tcp -d 192.168.100.2 --dport 22 -j ACCEPT
```

図 4 手順 4 の要件を満たすための iptables の編集コマンド

5. Windows から router へ通信を発生させ、server に一般権限ユーザでログインできることを確認する（必要に応じて known_hosts を編集して Host key 問題を解決する）。

図 5 はデスクトップ PC から router への ssh 接続を試みた結果である。



```
C:\Users\Ys090vgu>ssh user1@10.6.158.90
Warning: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ECDSA key sent by the remote host is
SHA256:6pfSRv/4YmznSNkajGtQFJM6s7uUvJM7BvUgWipNJR8.
Please contact your system administrator.
Add correct host key in C:\Users\Ys090vgu/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in C:\Users\Ys090vgu/.ssh/known_hosts:1
Host key for 10.6.158.90 has changed and you have requested strict checking.
Host key verification failed.
C:\Users\Ys090vgu>notepad .ssh\known_hosts
C:\Users\Ys090vgu>ssh user1@10.6.158.90
The authenticity of host '10.6.158.90 (10.6.158.90)' can't be established.
ECDSA key fingerprint is SHA256:6pfSRv/4YmznSNkajGtQFJM6s7uUvJM7BvUgWipNJR8.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.6.158.90' (ECDSA) to the list of known hosts.
user1@10.6.158.90's password:
# hostname
server
#
```

図 5 デスクトップ PC から router への ssh 接続に成功する様子

ssh コマンドを実行すると、”remote host identification has changed!” と警告が発生していることがわかる。一般に ssh のソフトウェアでは、~/ .ssh/known_hosts

に接続先のサーバの公開鍵情報を暗号化して保存しておく。次回接続した際には、ここに保存された公開鍵と照合することによって同じサーバかどうかを検証する[4]、このときに公開鍵の情報が一致しなければ、今回得られたような警告文が出力される。

今回の場合においては以前に windows から router に対して ssh 接続した際に router の公開鍵情報が `~/.ssh/known_hosts` に保存されており、今回も同じように router に対して ssh 接続のコマンドを実行したが、手順 4 において ssh のパケットを server に転送するよう設定したことで、server の公開鍵情報と以前に保存した router の公開鍵情報とで照合が行われ、一致せずに警告が発せられたと考えられる。したがってここでは記録されていた公開鍵情報を消去する、すなわち、`~/.ssh/known_hosts` の中身を空にすることで対処した。結果として、最終的に windows から server に user1 としてログインできていることが確認できる。

6. router 上で Wireshark を起動し、Windows ↔ router および router ↔ server の両方のインターフェースをキャプチャ対象に設定する。
7. Windows から router への SSH ログイン過程のパケットを router の Wireshark で解析し、iptables による転送（IP アドレスやポート番号の書き換え）が観測できるパケットを特定する。

図 6 は router 上で Wireshark を起動し、Windows ↔ router および router ↔ server の両方のインターフェースをキャプチャ対象に設定した様子である。両方のインターフェースを流れるパケットを比較すると、プロトコルやパケット内の情報が酷似していることが確認できる。実際に、1 つパケットを抽出して比較した例を図 7 に示す。

図 7 に示すパケットは、SSHv2 のパケットである。ネットワーク以下のプロトコルのヘッダ情報は server から router へのパケットであることを示す部分が router から windows へのパケットであることを示すものに置き換わっている一方で、tcp のヘッダにおいて、フィールドの値が完全一致しており、これらのパケットは、tcp とそれより上位のプロトコルのヘッダ、ひいてはデータにおいて同じ値が格納されている、すなわち、データが転送されていることが確認できる。

8. router の組織内側インターフェースに対して、server から組織外側ネットワークへの通信が可能になるように iptables を設定する（例：POSTROUTING で

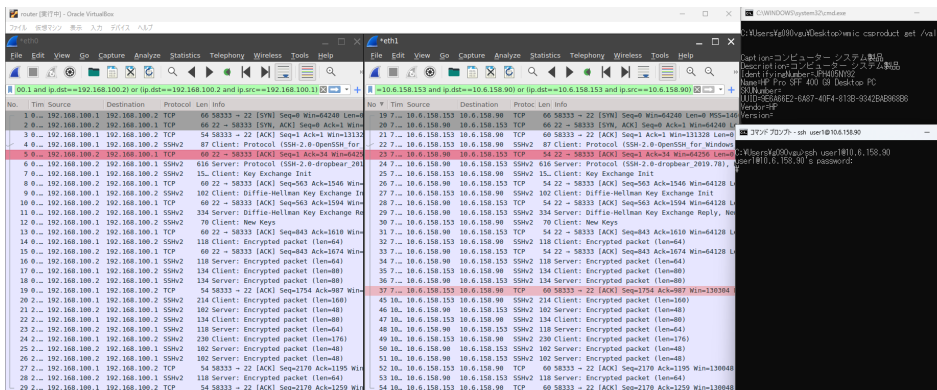


図 6 ssh ログイン時の Wireshark のキャプチャ結果

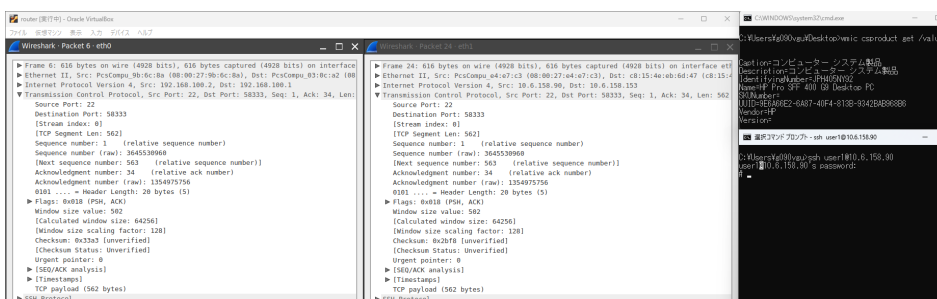


図 7 Windows ↔ router および router ↔ server を流れるパケットの比較例

SNAT/MASQUERADE を設定).

図 8 は, 手順 9 の要件を満たすための iptables の編集コマンドの実行結果である. 各コマンドの説明については手順 10 で行う.

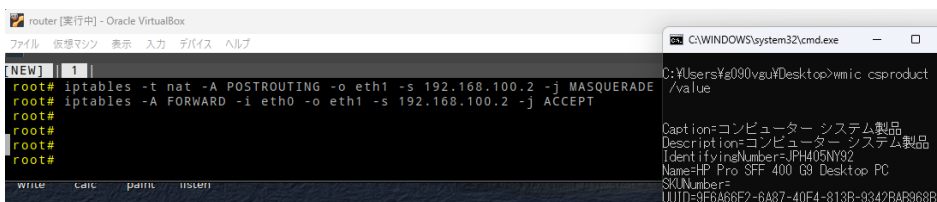
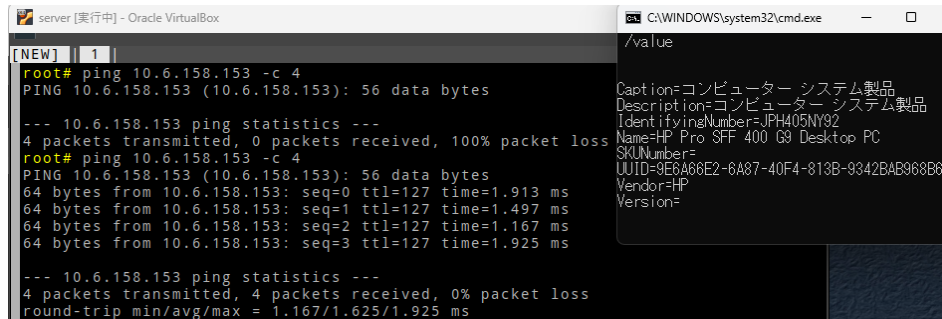


図 8 手順 9 の要件を満たすための iptables の編集コマンド

9. server から組織外ネットワークの端末へ通信を発生させ, router の Wireshark で解析し, iptables による変換 (送信元 IP の書き換え等) を示すパケットを特定する.

図 9 は、server から windows に対して ping コマンドを実行した結果である。

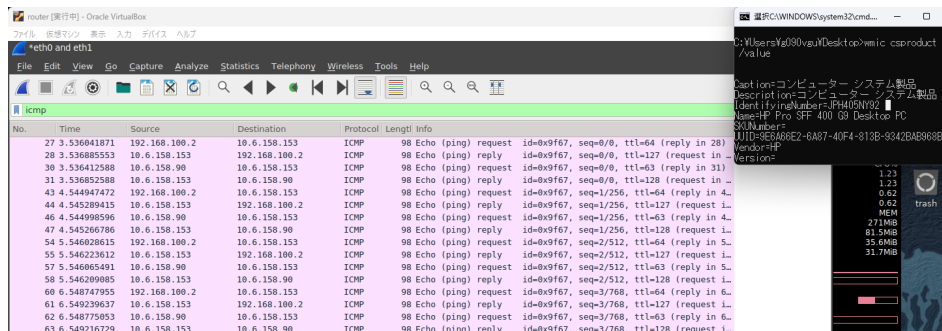


```
server [実行中] - Oracle VirtualBox
[NEW] 1
root# ping 10.6.158.153 -c 4
PING 10.6.158.153 (10.6.158.153): 56 data bytes
--- 10.6.158.153 ping statistics ---
4 packets transmitted, 0 packets received, 100% packet loss
root# ping 10.6.158.153 -c 4
PING 10.6.158.153 (10.6.158.153): 56 data bytes
64 bytes from 10.6.158.153: seq=0 ttl=127 time=1.913 ms
64 bytes from 10.6.158.153: seq=1 ttl=127 time=1.497 ms
64 bytes from 10.6.158.153: seq=2 ttl=127 time=1.167 ms
64 bytes from 10.6.158.153: seq=3 ttl=127 time=1.925 ms
--- 10.6.158.153 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 1.167/1.625/1.925 ms
```

図 9 server から windows に対する ping コマンドの実行結果

windows からの応答が確認でき、server と windows との間で通信が可能であることが確認できる。

続いて、図 10 に server から windows に対して ping コマンドを実行した際にキャプチャされたパケットの一覧を示す。



No.	Time	Source	Destination	Protocol	Length	Info
27	3.536841871	192.168.100.2	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=0/0, ttl=64 (reply in 28)
28	3.53685553	10.6.158.153	192.168.100.2	ICMP	98	Echo (ping) reply id=0x9f67, seq=0/0, ttl=127 (request in 28)
30	3.536422588	10.6.158.99	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=0/0, ttl=43 (reply in 31)
31	3.536852588	10.6.158.153	10.6.158.99	ICMP	98	Echo (ping) reply id=0x9f67, seq=0/0, ttl=128 (request in 30)
43	4.544947472	192.168.100.2	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=1/256, ttl=64 (reply in 4)
44	4.545289415	10.6.158.153	192.168.100.2	ICMP	98	Echo (ping) reply id=0x9f67, seq=1/256, ttl=127 (request in 43)
46	4.544985596	10.6.158.99	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=1/256, ttl=63 (reply in 4)
47	4.545266786	10.6.158.153	10.6.158.99	ICMP	98	Echo (ping) reply id=0x9f67, seq=1/256, ttl=128 (request in 46)
54	5.546028615	192.168.100.2	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=2/512, ttl=64 (reply in 5)
55	5.546223612	10.6.158.153	192.168.100.2	ICMP	98	Echo (ping) reply id=0x9f67, seq=2/512, ttl=127 (request in 54)
57	5.546065491	10.6.158.99	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=2/512, ttl=63 (reply in 5)
58	5.546209885	10.6.158.153	10.6.158.99	ICMP	98	Echo (ping) reply id=0x9f67, seq=2/512, ttl=128 (request in 57)
60	6.548747955	192.168.100.2	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=3/768, ttl=64 (reply in 6)
61	6.549239637	10.6.158.153	192.168.100.2	ICMP	98	Echo (ping) reply id=0x9f67, seq=3/768, ttl=127 (request in 60)
62	6.548775853	10.6.158.99	10.6.158.153	ICMP	98	Echo (ping) request id=0x9f67, seq=3/768, ttl=63 (reply in 6)
63	6.549216729	10.6.158.153	10.6.158.99	ICMP	98	Echo (ping) reply id=0x9f67, seq=3/768, ttl=128 (request in 62)

図 10 server から windows に対する ping コマンド実行時にキャプチャされたパケット一覧

10. 実験結果として、デスクトップ PC から server へログインできること、server から組織外ネットワークへ通信できることを示し、使用したすべての iptables 設定を列挙する。各設定がどのようにパケットを書き換え、転送処理を実現しているかを具体的に説明してレポートにまとめる。

手順 5, 9 においてそれぞれ転送によってデスクトップ PC から server にログインできることと組織外ネットワークに属する装置と通信ができること

図 11, 12 に手順 4, 8 における iptables の状態を示す。

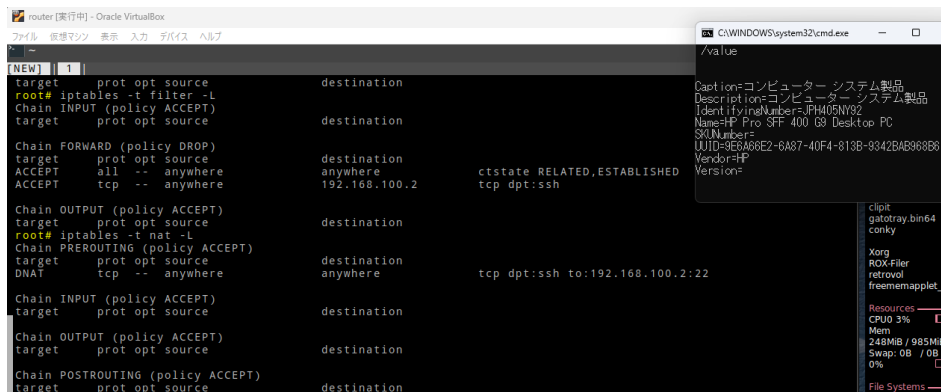


図 11 手順 4 における iptables の状態

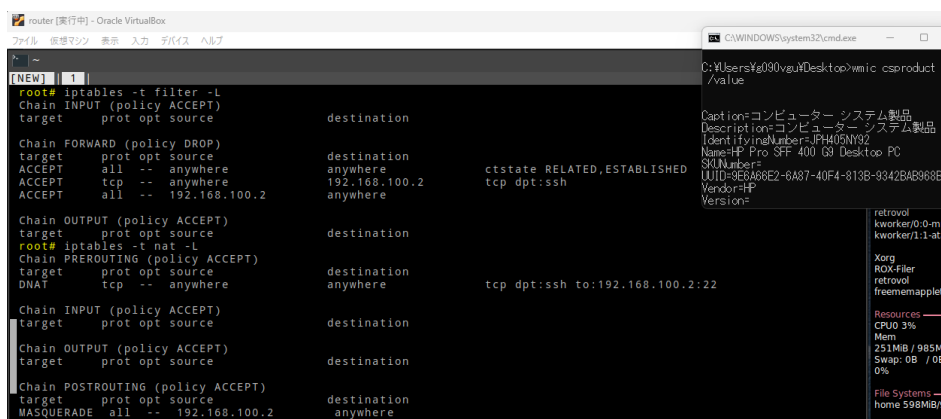


図 12 手順 8 における iptables の状態

まず 4 手順 4 で行った設定について説明する。

まず sysctl コマンド [1] を -w オプションで用いて, ipv4.ip.forward の値を 1 に設定することで, ポートフォワーディングを有効化した。

次に tcp22 番ポートは以外へのアクセスは転送しないようにするため, -P オプションによって対象のチェーンのポリシーを設定している。ここでは filter テーブル上の FORWARD チェインに対して DROP というポリシーを設定し, デフォルトですべてのパケットを転送できないよう設定している [3]。

続いて iptables における接続追跡機能である conntrack の -ctstate オプションで ESTABLISHED と RELATED の statelist を指定してそれらの statelist に該当するパケットの通信を許可することによって [2], server から windows への帰りのパケット (tcp22 番ポートあて以外のパケット) を転送するための設定をしている。

次に nat テーブルの PREROUTING チェインに対して eth1 の 10.6.158.0/24 のネットワークから自身に対して送信された tcp22 番ポートへのパケットを宛先が 192.168.100.2 のパケットへと書き換える設定を行っている。

参考文献

- [1] kazmax. sysctl - システム管理コマンドの説明 - Linux コマンド集 一覧表. kazmax - Linux で自宅サーバー. Accessed: 2026-06-04. 2022. URL: <http://kazmax.zpp.jp>.
- [2] Linux man-pages project. iptables-extensions(8) — Linux manual page. iptables version 1.8.13. Accessed: 2026-06-04. man7.org. 2024. URL: <https://man7.org/linux/man-pages/man8/iptables-extensions.8.html>.
- [3] Rusty Russell and Netfilter Core Team. iptables(8) — IPv4/IPv6 のパケットフィルタと NAT の Version 1.4.21. Linux Manual Page, Section: iptables 1.4.21 (8). Netfilter Project. 2013. URL: <https://linuxjm.sourceforge.io/html/iptables/man8/iptables.8.html>.
- [4] S.O. known_hosts とは？. Zenn. Accessed: 2026-06-04. May 2025. URL: <https://zenn.dev>.